

PRODUCT REQUIREMENTS DOCUMENT

Controlled Document Approval System

A small, correctness-first workflow application for controlled authoring, review, publication, archival, and append-only audit history.

Product: ElevateBox Engineering Challenge

Document type: Implementation PRD

Preferred stack: SvelteKit 5, TypeScript, PostgreSQL, Drizzle ORM

Delivery scope: Local full-stack build

Source of truth: ElevateBox_SWE_Challenge.pdf

1. Product Summary

The system controls how documents move from private drafts to public publication. A document must never become published unless an authorized reviewer or administrator publishes an approved document. Every important write must produce an audit event, and state changes and their audit events must commit atomically.

1.1 Product objective

Deliver a small, locally runnable full-stack application that demonstrates correct server-side authorization, explicit workflow modelling, database-backed integrity, transactional audit history, and protection against stale writes.

1.2 Success definition

- All nine user stories are demonstrable through a minimal interface.
- All non-negotiable integrity rules are enforced by the server.
- Invalid, unauthorized, and stale operations fail clearly without corrupting document state or history.
- A reviewer can run the app locally by following the README and using the seeded accounts.

2. Scope

2.1 In scope

- Seeded-user login and logout with a simple session.
- Create and edit owned draft or rejected documents.
- Submit documents for review.
- Reviewer approval and rejection, with mandatory rejection comments.
- Publication of approved documents by a reviewer or administrator.
- Viewer access to published documents only.
- Chronological, append-only audit history.
- Optimistic concurrency control using an expected version.
- Administrative archival without deleting database records or history.

2.2 Explicitly out of scope

- Signup, password reset, email verification, email sending, and OAuth.
- File upload and rich-text editing.
- Complex administration dashboards.
- Real deployment or infrastructure provisioning.
- Pixel-perfect styling and mobile-perfect layouts.
- Restoring archived documents unless separately designed and explained; restoration is not required.
- Direct editing of published documents or automatic document version history.

2.3 Guiding principle

The system is defined as much by the actions it refuses as by its happy path. Client-side visibility improves usability, but every permission, transition, validation rule, and visibility boundary must be enforced on the server.

3. Users and Roles

Role	Primary capability	Required limitation
Viewer	View published documents	Cannot access private workflow documents or mutations
Author	Create, edit, submit, and reopen own documents	Cannot edit others' documents or review/publish
Reviewer	Review submitted documents; publish approved documents	Cannot approve own documents
Admin	Manage and archive documents; publish approved documents	Archiving must preserve records and history

3.1 Seeded accounts

Email	Role
alice@example.com	author
bob@example.com	reviewer
admin@example.com	admin
viewer@example.com	viewer

4. Document Workflow

The server must treat the following state machine as the authoritative workflow. A move not listed here is invalid and must not silently succeed.

Current state	Action	New state	Authorized actor	Additional rule
draft	submit	submitted	Owning author	Title and body must be non-empty
submitted	approve	approved	Reviewer	Reviewer cannot approve own document
submitted	reject	rejected	Reviewer	Non-empty comment required
rejected	reopen	draft	Owning author	Returns document to editable draft workflow
approved	publish	published	Reviewer or admin	Only approved documents may be published
draft / submitted / approved / published	archive	archived	Admin	Record remains in database and history

4.1 State behavior

- Draft and rejected documents are editable only by their owner.
- Submitted documents are read-only to authors and appear in the reviewer queue.
- Approved documents may be published but are not directly editable.
- Published documents are publicly visible to authenticated viewers but not directly editable.
- Archived documents are terminal for the required workflow, excluded from active queues, and retained with full history.

5. Functional Requirements

FR-01 — Log in as a seeded user

- The login screen lets a user select or identify a seeded account.
- The application displays the current user's name and role.
- The server derives permissions from the authenticated session.
- Unauthenticated users cannot access private pages or invoke private actions.

FR-02 — Create a draft

- An author can create a document with title and body.
- The new document starts in draft and is owned by its creator.
- Empty title or body is rejected.
- Creation records an audit event.

FR-03 — Edit a document

- The owning author can edit title and body in draft or rejected state.
- Every successful edit records an audit event.
- Documents in submitted, approved, published, or archived state are not editable through this workflow.

FR-04 — Submit for review

- The owning author can move a valid draft to submitted.
- The document becomes read-only to the author and appears in the reviewer queue.
- Submission records an audit event.

FR-05 — Review a submission

- A reviewer can inspect submitted documents.
- A reviewer can approve or reject a submitted document.
- A rejection requires a non-empty comment.
- A reviewer cannot approve their own document.
- Approval and rejection record audit events.

FR-06 — Publish an approved document

- A reviewer or administrator can publish an approved document.
- A published document becomes visible to viewers.
- Publication records an audit event.
- No other state may be published.

FR-07 — View audit history

- A user with document access can view its chronological history.
- Events include actor, action, timestamp, relevant comment or metadata, and previous/new status where applicable.
- History includes created, edited, submitted, approved, rejected, published, and archived events as applicable.

FR-08 — Handle concurrent updates

- Each mutation supplies the version the client expects.
- A stale mutation fails with a conflict response and does not overwrite the newer state.
- The UI shows a clear refresh/reload message.
- In the Bob/Carol scenario, Bob's approval remains final when Carol later submits a stale rejection.

FR-09 — Archive a document

- Only an administrator can archive an eligible document.
- Archived documents leave active queues but remain stored.
- Archival records an audit event.

- Archived documents are not editable or publishable through normal workflows.

6. Authorization and Visibility Requirements

Capability	Viewer	Author	Reviewer	Admin
View published	Yes	Yes	Yes	Yes
View private workflow document	No	Own documents	Review-access documents	Management access
Create draft	No	Yes	No	Not required
Edit draft/rejected	No	Own only	No	Not required
Submit / reopen	No	Own only	No	No
Approve / reject	No	No	Submitted only; not own approval	No
Publish	No	No	Approved only	Approved only
Archive	No	No	No	Eligible states only

All access checks must run on the server for both page/data loads and mutations. A hidden or disabled UI control is not an authorization mechanism.

7. Data Requirements

Entity	Required fields	Purpose
User	ID, name, email, role	Authenticated actor and authorization identity
Session	Session identifier, user ID, expiry	Simple seeded-user authentication
Document	ID, title, body, status, author ID, version, timestamps	Database source of truth for content and workflow state
Audit event	ID, document ID, actor ID, action, timestamp, comment/metadata, previous status, new status	Append-only record of important actions and state changes

7.1 Data invariants

- Every document has an owner and a valid workflow status.
- Document versions increase on successful mutations.
- Every state-changing operation produces exactly one matching audit event in the same transaction.
- Audit history is preserved when a document is archived.
- Published content is never silently modified through the draft editing flow.

8. Transaction and Concurrency Requirements

8.1 Atomic writes

Each state-changing service operation must validate authentication, authorization, input, current state, and expected version, then update the document and append the audit event within one database transaction. Failure at any point rolls back the complete operation.

8.2 Optimistic concurrency

Every mutation includes the client's expected document version. The update must conditionally match the document ID, expected version, and required current state. If no record matches, the server returns a conflict and creates no audit event. The version check belongs within the transactional write path so validation and mutation cannot drift apart.

8.3 Required conflict scenario

1. Bob and Carol open the same submitted document at the same version.
2. Bob approves it successfully; its status and version change and an audit event is appended.
3. Carol submits a rejection using the stale version.
4. Carol receives a clear conflict response; Bob's approval and audit history remain unchanged.

9. Interface Requirements

The interface should be deliberately simple. Its purpose is to make every required workflow and refusal case easy to demonstrate; visual polish is secondary to clarity and correctness.

View	Required content/actions
Login	Seeded accounts and login action
Dashboard / document list	Role-appropriate active documents and published documents
Create/edit document	Title and body fields, validation, save action, current status/version
Document detail	Content, owner, state, version, permitted actions, rejection comment where relevant
Reviewer queue	Submitted documents available for review
Published list/detail	Published documents visible to viewers
Audit history	Chronological actor/action/time/status/comment timeline
Archive view	Archived records available to authorized users, absent from active queues

9.1 User feedback

- Validation errors identify missing or invalid input.
- Forbidden operations show a clear authorization message without exposing private content.
- Invalid transitions identify that the document state has changed or the requested action is unavailable.
- Stale updates show a conflict message instructing the user to refresh before acting again.

10. Technical Requirements

Layer	Required/preferred choice
Framework	SvelteKit with Svelte 5
Language	TypeScript
Database	PostgreSQL
Database toolkit	Drizzle ORM
Authentication	Simple seeded login and server-side session; production-grade auth is not required
API layer	SvelteKit server routes/actions; Hono only if useful
Infrastructure	None required; local execution is sufficient

The database is the source of truth. Domain workflow and authorization logic should be centralized and reused by all server entry points.

11. Error and Failure Behavior

Failure	Expected result
Unauthenticated private request	Rejected; no mutation
Unauthorized role or ownership	Rejected on the server; no mutation or audit event
Invalid input	Rejected with useful validation feedback
Invalid transition	Rejected; state remains unchanged
Missing rejection comment	Rejected; document stays submitted
Stale expected version	Conflict response; newer state remains intact
Audit insertion failure	Entire document mutation rolls back
Archived-document mutation	Rejected through normal workflow

12. Acceptance Criteria

- A seeded user can log in, see their identity and role, and log out.
- Unauthenticated users cannot open private pages or perform private actions.
- An author can create, edit, submit, and reopen only their own eligible documents.
- A reviewer can approve or reject only submitted documents, and rejection requires a comment.
- An author cannot approve their own document.
- Only an approved document can become published, and only a reviewer or administrator can publish it.
- A viewer can access published documents and cannot reach non-published documents through UI, guessed URL, or direct request.
- Every successful state change has a matching audit event committed in the same transaction.
- A failed state change leaves both document state and audit history unchanged.
- A stale mutation returns a conflict and cannot overwrite a newer mutation.
- Only an administrator can archive; archive removes the record from active queues without deleting it or its history.
- The repository includes runnable setup instructions, environment variables, migration/seed commands, tests, and seeded login credentials.

13. Verification Strategy

Testing should concentrate on service and integration behavior because the evaluation prioritizes workflow correctness, authorization, transactions, audit consistency, and concurrency over visual design.

13.1 Required integration coverage

- Authentication and private route protection.
- Role, ownership, and state checks for every mutation.
- Every permitted and forbidden workflow transition.
- Mandatory rejection comments and self-approval prevention.
- Viewer visibility boundaries for list, detail, and direct request paths.
- Atomic rollback when either document mutation or audit write fails.
- Bob/Carol stale-update conflict behavior.
- Archive retention and exclusion from active queues.

13.2 Manual demonstration path

Author creates and edits a draft, submits it, reviewer approves it, reviewer or administrator publishes it, viewer opens it, and an authorized user verifies the complete audit history. A second run demonstrates rejection, reopen, resubmission, stale conflict, and archival.

14. Documentation and Submission Requirements

14.1 README

- Prerequisites, environment variables, installation, database migration, seed, run, and test commands.
- Seeded login credentials and concise architecture overview.

14.2 DESIGN.md

- Most important system invariants.
- Database-enforced versus application-enforced invariants.
- Permission model and visibility boundaries.
- Stale/conflicting update prevention.
- Transaction and audit consistency design.
- Considered failure cases.
- Improvements with more time and changes required for production.

14.3 Submission readiness

- The application starts successfully from a clean local setup using the README.
- Shared repository and recording links, if used, are accessible without the evaluator being signed in as the candidate.

15. Product Priorities

Priority	Area	Rationale
P0	Server authorization and visibility	Prevents unauthorized access and actions
P0	State-machine enforcement	Prevents invalid workflow state
P0	Transactional audit consistency	Prevents untraceable document changes
P0	Optimistic concurrency	Prevents stale overwrites
P1	Minimal workflow UI	Makes required behavior demonstrable
P1	Integration tests and design note	Provides evidence and explains engineering judgment
P2	Visual refinement	Useful only after correctness is complete